CURSO DE FORMACIÓN · 20 HORAS

Big Data e IA

Bloque 3: sistemas y arquitecturas Big Data



3 horas · PySpark · Databricks

PLAN DE LA SESIÓN

Tres horas, tres partes

PARTE 1 · ORIENTATIVO 1 H 35 MIN

Ideas clave

- Apertura: seis casos de entrevista (25 min)
- Historia y de pandas a Spark
- Arquitectura y nodos
- Plan de ejecución y linaje
- Databricks
- Opcional: Flink, configuración, Databricks por dentro

PARTE 2 · ORIENTATIVO 1 H 05 MIN

Práctica guiada

- Capturar y cargar el flujo de Wikipedia
- Filtros, agregaciones, joins y ventanas
- SQL, cache y guardado
- Streaming: ver crecer la tabla
- Errores típicos del notebook

PARTE 3 · ORIENTATIVO 20 MIN

Ejercicio

- Cuatro retos con PySpark
- Puesta en común

03

Sistemas y arquitecturas Big Data

Bloque 3 · 3 horas

APERTURA DEL BLOQUE

Seis preguntas de entrevista

Antes de ver Spark: ¿qué responderíais en una entrevista?

1 · ¿Pandas o Spark?

Un CSV de 2 GB y un informe semanal.

2 · El shuffle

Un informe que pasa de 20 minutos a 3 horas.

3 · Un join desigual

500 GB contra 5 MB.

4 · La caché que frena

Cachear y que todo vaya más lento.

5 · Ficheros pequeños

Diez mil ficheros al día.

6 · Alerta o informe

Una media móvil: ¿lotes o flujo?

APERTURA · CASO 1 DE 6

¿Pandas o Spark?

ESCENARIO (FICTICIO)

Una tienda online guarda los pedidos del mes en un CSV. Cada lunes, marketing pide un informe de ventas por categoría. Hoy lo hace un analista con pandas en su portátil.

2 GB
tamaño del CSV

12 M
filas de pedidos

16 GB
memoria del portátil

LA PREGUNTA

¿Lo pasarías a Spark?

PENSAD EN…

- Cuánto va a crecer el fichero y cada cuánto se procesa
- Cuánta memoria gasta pandas al cargarlo
- Quién lo mantiene y cuánto cuesta un clúster

APERTURA · CASO 1: UNA BUENA RESPUESTA

Depende, y se razona

No hay un sí o un no: hay criterios.

Hoy: probablemente no

2 GB caben en una máquina. Pandas es más simple, arranca al momento y no hay clúster que pagar ni mantener.

Ojo con la memoria

Al cargar un CSV, pandas puede ocupar bastante más memoria que el fichero. En 16 GB va justo si el fichero crece.

Qué cambiaría la respuesta

Que pase a decenas o cientos de GB, que lleguen datos de muchas fuentes o que varios equipos compartan las mismas tablas.

Pasos intermedios

Guardar en Parquet en vez de CSV, leer solo las columnas necesarias o procesar por trozos antes de saltar a un clúster.

APERTURA · CASO 2 DE 6

El informe que no termina

ESCENARIO (FICTICIO)

Una cadena de supermercados calcula las ventas por tienda y día con sus tickets del último año, en Parquet. Al añadir un join con la tabla de clientes y un distinct, el informe pasa de minutos a horas.

3.000 M
líneas de ticket

20 min
antes

3 h
después

LA PREGUNTA

¿Qué es el shuffle y por qué es tan caro?

PENSAD EN…

- Por dónde viajan los datos entre nodos
- Qué operaciones lo provocan
- Cómo reducirlo sin cambiar el resultado

APERTURA · CASO 2: UNA BUENA RESPUESTA

Mover datos es lo caro

Cada Exchange del plan es un viaje de datos por la red.

Qué es

Redistribuir los datos entre nodos para agruparlos por clave: cada nodo escribe en su disco un trozo para cada tarea siguiente, y esas tareas los leen por la red.

Por qué es caro

Disco, red y espera: la etapa siguiente no empieza hasta que acaba la anterior. En el plan aparece como Exchange.

Qué lo provoca

groupBy, join, distinct, orderBy y repartition. En el ejemplo, el join y el distinct añaden shuffles nuevos.

Cómo reducirlo

Filtrar y quedarse con las columnas necesarias antes; broadcast si una tabla es pequeña; evitar distinct y orderBy innecesarios; dejar que la ejecución adaptativa ajuste las particiones.

APERTURA · CASO 3 DE 6

Un join muy desigual

ESCENARIO (FICTICIO)

Una plataforma de reparto une sus pedidos con la tabla de zonas de reparto por zona_id para calcular tiempos medios. El join tarda dos horas y el plan muestra un shuffle enorme.

500 GB
tabla de pedidos

5 MB
tabla de zonas

2 h
duración del join

LA PREGUNTA

¿Qué harías para acelerarlo?

PENSAD EN…

- Qué cuesta más: mover la tabla grande o la pequeña
- Cómo decide Spark qué tabla es pequeña
- Qué puede salir mal si te equivocas

APERTURA · CASO 3: UNA BUENA RESPUESTA

Enviar la pequeña, no mover la grande

python

```python
from pyspark.sql import functions as F

resultado = pedidos.join(
    F.broadcast(zonas),
    on="zona_id", how="left")

resultado.explain()
# BroadcastHashJoin en lugar de SortMergeJoin
```

Qué hace

Cada executor recibe la tabla de zonas; los 500 GB de pedidos no viajan por la red.

Umbral

Automático si la tabla estimada pesa menos de 10 MB (por defecto); F.broadcast lo fuerza.

Riesgo

Si la «pequeña» no lo es, falta memoria en el driver o los executors.

APERTURA · CASO 4 DE 6

La caché que frena

ESCENARIO (FICTICIO)

Una analista de una aseguradora carga los siniestros, ejecuta df.cache() «por si acaso» y lanza una sola agregación. El job tarda el doble que sin caché y el clúster va justo de memoria.

300 GB
siniestros cargados

1
uso del DataFrame

×2
tiempo con caché

LA PREGUNTA

¿Cómo puede ir más lento con caché?

PENSAD EN…

- Qué trabajo extra hace cache() la primera vez
- Cuánta memoria queda para ejecutar
- Cuántas veces se reutiliza el DataFrame

APERTURA · CASO 4: UNA BUENA RESPUESTA

Cachear no es gratis

La caché es una apuesta: solo gana si reutilizas el DataFrame.

Cuesta trabajo

La primera acción lee, procesa y guarda los datos en memoria (y en disco si no caben). Si solo hay una pasada, es trabajo de más.

Memoria compartida

La caché y la ejecución usan la misma región de memoria: si la ejecución necesita sitio, parte de la caché se expulsa o se mueve a disco.

Cuándo sí

Cuando el mismo DataFrame, ya filtrado y con las columnas necesarias, se usa varias veces.

Cómo hacerlo bien

Cachear después de filtrar y seleccionar, comprobar que cabe, liberar con unpersist al terminar o guardar el resultado intermedio en Parquet o Delta.

APERTURA · CASO 5 DE 6

Diez mil ficheros al día

ESCENARIO (FICTICIO)

Una app de movilidad escribe cada minuto lo que llega en Parquet, con una carpeta por ciudad. Tras unas semanas, leer el histórico es muy lento y el almacenamiento está lleno de ficheros diminutos.

10.000
ficheros al día

200 KB
tamaño medio

1 min
frecuencia de escritura

LA PREGUNTA

¿Es un problema? ¿Cómo lo arreglas?

PENSAD EN…

- Qué cuesta abrir un fichero frente a leerlo
- Dónde puedes actuar: al escribir o después
- Qué tamaño de fichero sería razonable

APERTURA · CASO 5: UNA BUENA RESPUESTA

Menos ficheros, más grandes

python

```python
# Al escribir: menos particiones de salida
(df.coalesce(8)
   .write.mode("append")
   .partitionBy("ciudad")
   .parquet(destino))

# Escribir menos veces: cada 10 minutos
# y no cada minuto
```

Sí es un problema

Cada fichero se lista y se abre: lectura lenta y tareas diminutas.

Al escribir

coalesce o repartition antes de guardar; con coalesce(1) escribe una sola tarea.

Después

Compactar (en Delta, OPTIMIZE) y no particionar por columnas con muchos valores.

APERTURA · CASO 6 DE 6

Alerta o informe

ESCENARIO (FICTICIO)

Un supermercado tiene cámaras frigoríficas que envían la temperatura cada 10 segundos. Hay que avisar al encargado si la media de los últimos 3 minutos supera los 6 °C, y entregar un informe diario para la auditoría sanitaria.

400	10 s	3 min
cámaras	una lectura	ventana de la media

LA PREGUNTA

¿Lotes o flujo para cada necesidad?

PENSAD EN…

- Cuánto vale una alerta que llega tarde
- Quién espera el resultado y cuándo
- Qué cuesta tener algo siempre encendido

APERTURA · CASO 6: UNA BUENA RESPUESTA

La latencia decide

La misma media, dos formas de calcularla.

La alerta: flujo

Una ventana deslizante de 3 minutos que avanza cada minuto, sobre los datos según llegan. Se avisa en segundos o minutos.

El informe: lotes

La misma media sobre el día completo, una vez al día. Es más barato y más sencillo de repetir y auditar.

Un solo motor

En Spark, la media móvil se escribe con window() tanto en lotes como en flujo; solo cambia cómo se lee y se escribe el DataFrame.

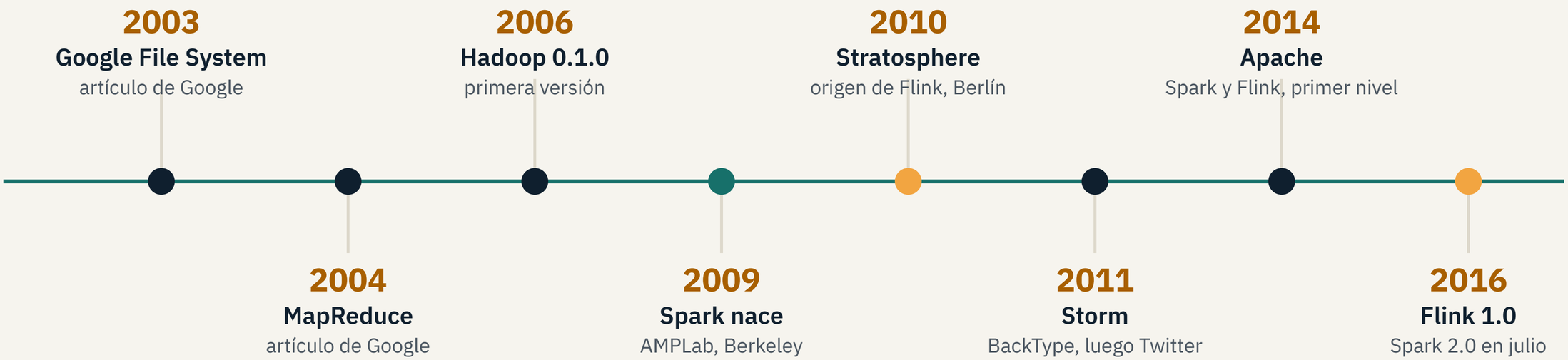
Lo que hay que prever

Lecturas que llegan tarde, el estado que se guarda entre microlotes y el coste de tener la consulta siempre encendida.

El mismo análisis, otro motor

	PANDAS	SPARK
Dónde se ejecuta	En una sola máquina	En un clúster de máquinas
Tamaño de datos	Lo que cabe en la memoria	Muy por encima de una máquina
Cuándo calcula	Al momento, línea a línea	Cuando pedimos un resultado
Sintaxis	df[df['team'] == ...]	df.filter(F.col('team') == ...)
Uso típico	Análisis y prototipos	Procesar grandes volúmenes

Antes de Spark: cómo llegamos hasta aquí



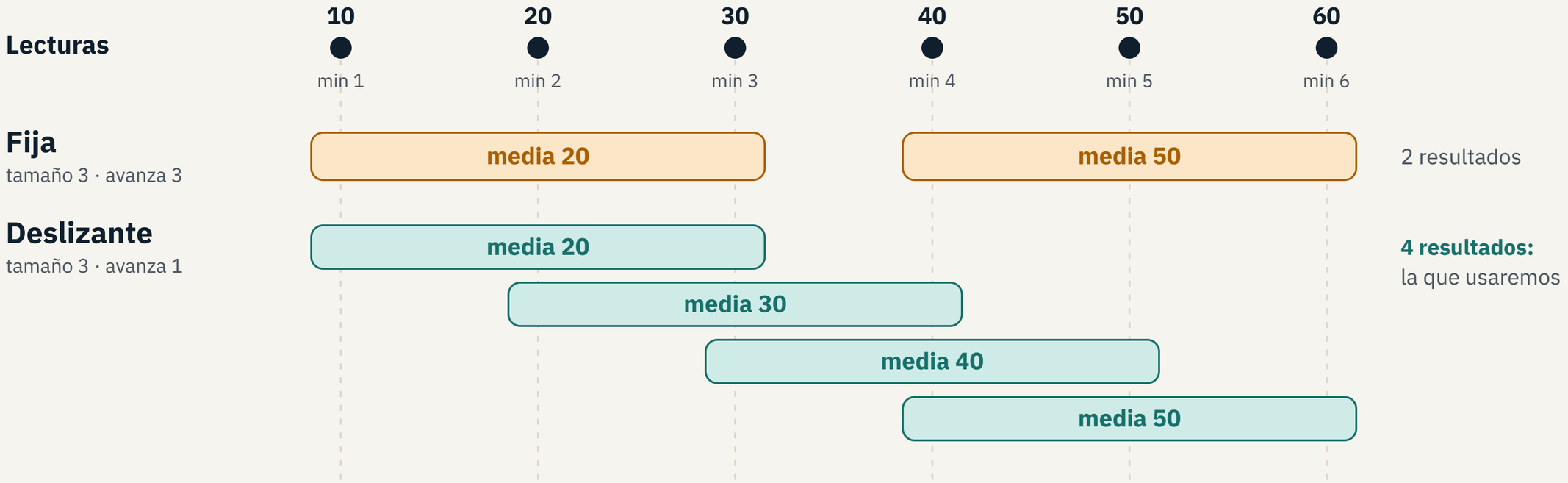
Primero se resolvió guardar y procesar mucho; después, hacerlo más rápido y casi en tiempo real.

Cuatro motores, cuatro ideas

Lo que cambia entre ellos es cómo reparten el trabajo en el tiempo.

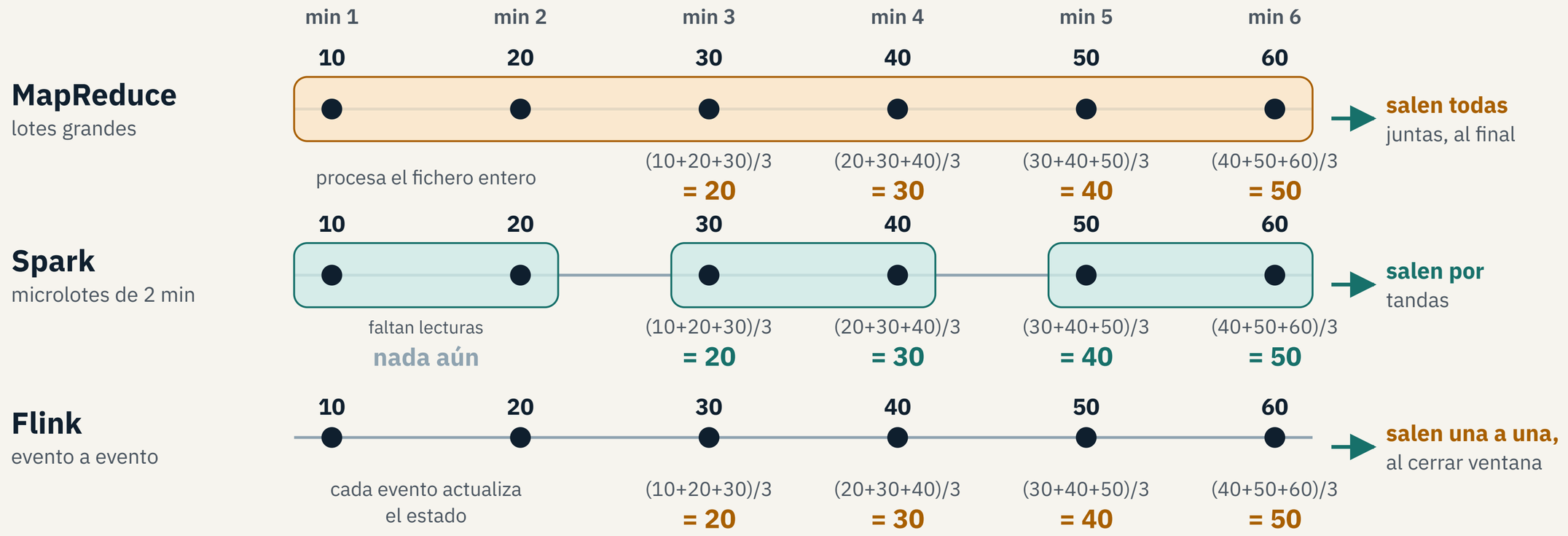
	HADOOP MAPREDUCE	STORM	SPARK	FLINK
Modelo	Lotes que pasan por disco	Evento a evento	Lotes en memoria	Evento a evento
Nace en	Google 2004, Hadoop 2006	BackType, 2011	Berkeley, 2009	Berlín, 2010
Punto fuerte	Mucho volumen, fiable	Poca espera	Un motor para todo	Streaming con estado

Qué es una ventana



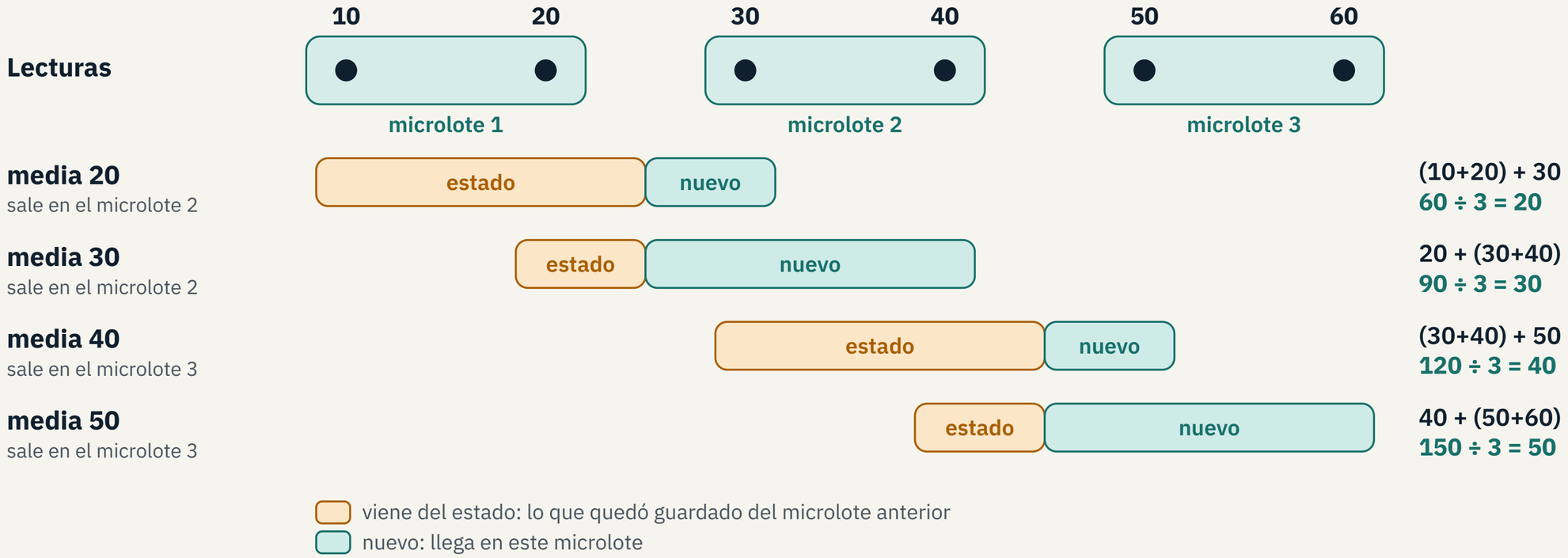
Ventana = las lecturas que entran en cada cálculo. Se define por su tamaño y por cuánto avanza.

Un mismo cálculo, tres modelos: la media móvil



Cada media usa las 3 últimas lecturas. Es la misma en los tres; cambia cuándo sale.

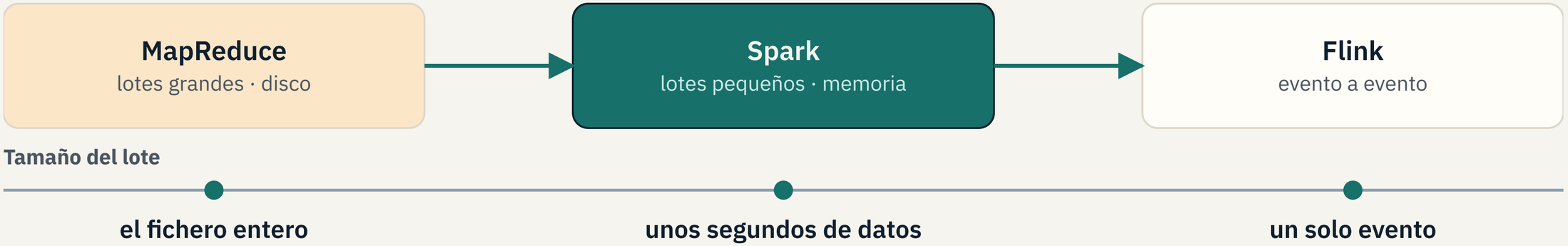
Spark: cada media cruza dos microlotes



Spark guarda en el estado lo que falta de cada ventana y lo completa con el microlote siguiente.

Spark: el puente entre MapReduce y Flink

Spark junta el mundo de los lotes con el de los flujos.



Hereda de MapReduce

Trabaja por lotes y reparte el trabajo en tareas; si una falla, se repite solo esa parte.

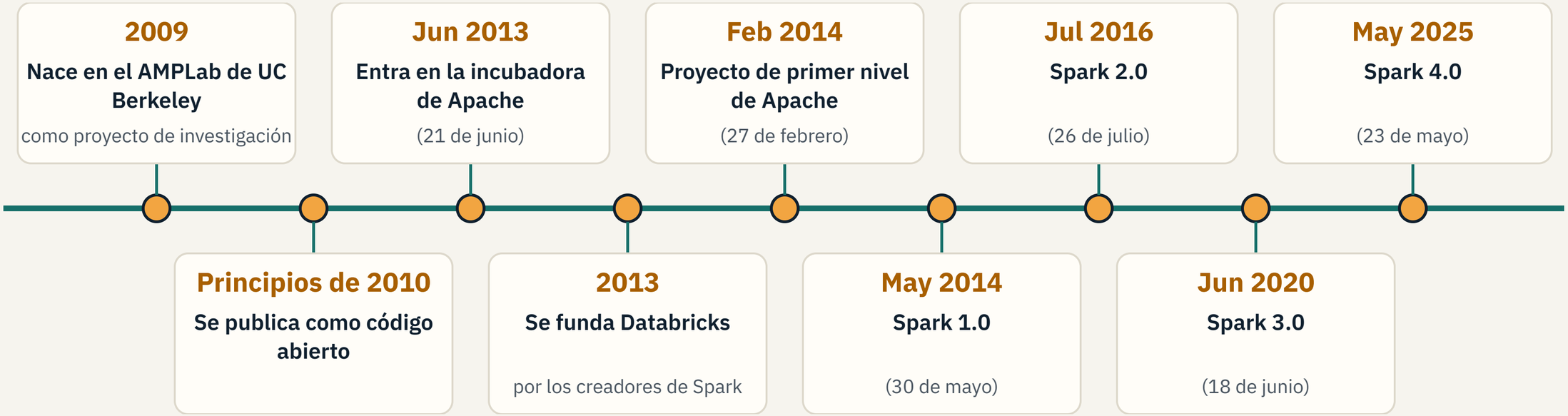
Aporta

Datos en memoria entre pasos y una API común, el DataFrame, para lotes y flujos: el mismo código sirve para los dos.

Se acerca a Flink

Con microlotes pequeños llega a latencias de unos 100 ms; para esperar menos hay que procesar evento a evento.

De Berkeley a Apache: una línea de tiempo



Empezó como investigación de un doctorando de Berkeley y hoy es un proyecto de la fundación Apache.

Qué aportó frente a MapReduce

	MAPREDUCE (BLOQUE 1)	SPARK
Pasos intermedios	Suelen pasar por disco	Pueden quedarse en memoria
Programación	Funciones map y reduce	DataFrames, SQL y funciones de alto nivel
Fallos	Se vuelve a ejecutar la tarea	Se recalculan las particiones perdidas con el linaje
Alcance	Procesamiento por lotes	Un motor para SQL, streaming, ML y grafos
Evaluación	Cada trabajo se lanza explícitamente	Plan perezoso: se optimiza entero antes de ejecutar

Repartir, procesar en paralelo, combinar

1 · Repartir

Los datos se dividen en trozos (particiones) y se reparten entre las máquinas del clúster.

2 · Procesar

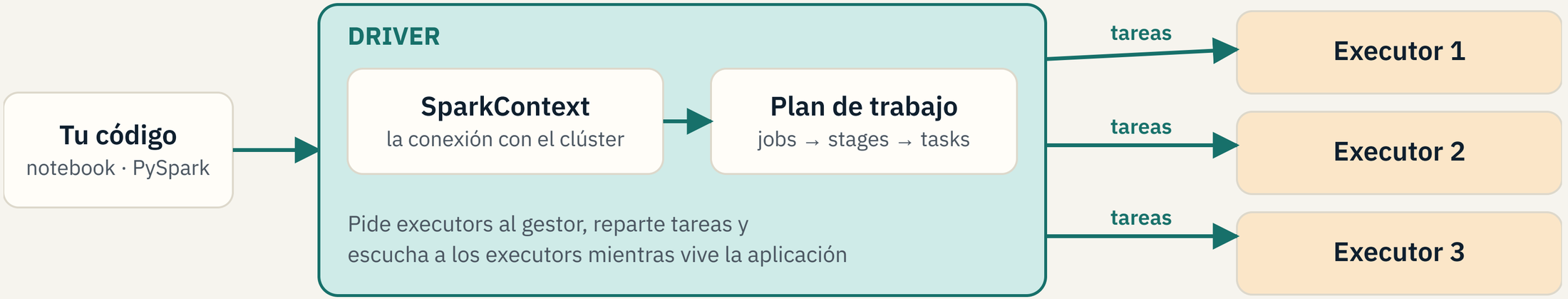
Cada máquina trabaja a la vez sobre su parte, sin esperar a las demás.

3 · Combinar

Los resultados parciales se unen para dar el resultado final.

La misma idea que **MapReduce** del bloque 1, con más comodidad.

Driver: el que coordina



Recibe y planifica

Convierte las operaciones de nuestro código en un plan de jobs, stages y tasks.

Reparte tareas

Envía cada tarea a un executor y recoge los resultados.

Es un punto único

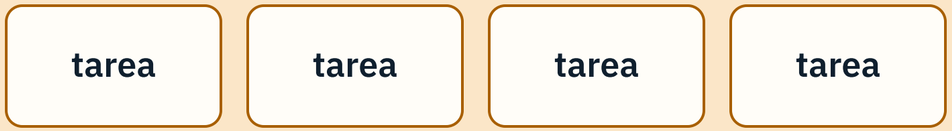
Si el driver cae, la aplicación termina. Los resultados que traemos con collect() caben en su memoria.

Executors: los que trabajan

Worker: una máquina del clúster

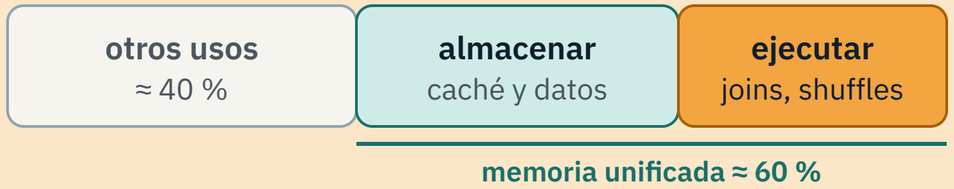
EXECUTOR

Hilos de tareas (spark.executor.cores)



cada tarea trabaja sobre una partición

Memoria (spark.executor.memory, 1g por defecto)



Ejecuta tareas

En varios hilos a la vez: tantos como núcleos (cores) tenga asignados.

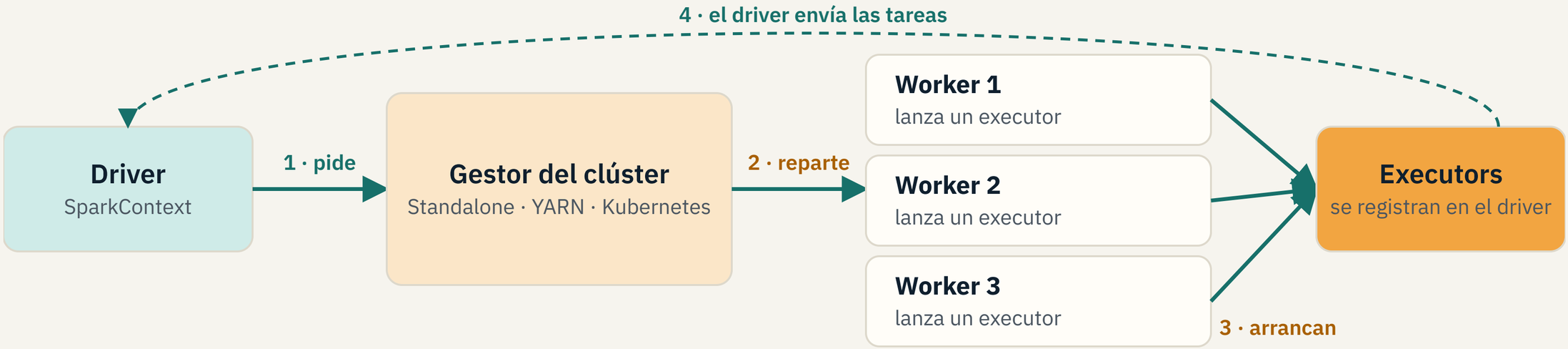
Guarda datos

En memoria, o en disco si no caben, cuando usamos cache() o persist().

Vive con la aplicación

Cada aplicación tiene sus propios executors, aislados de los de otras.

Gestor del clúster: el que reparte



Qué se le pide

Cuántos executors y con cuánta memoria y cuántos núcleos cada uno.

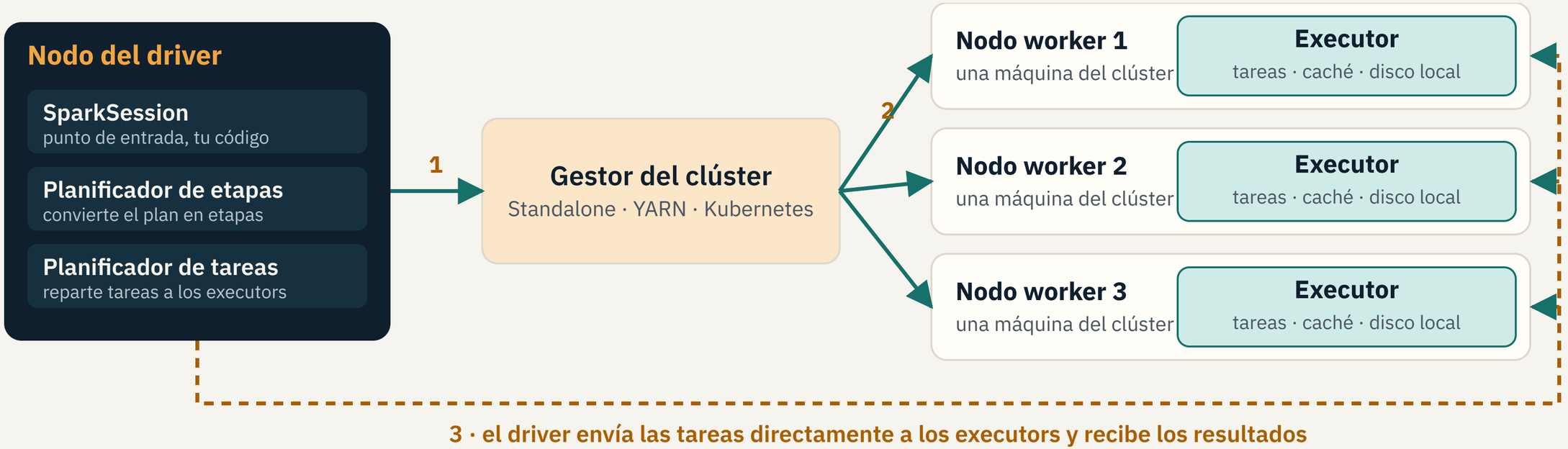
Asignación fija o dinámica

spark.dynamicAllocation.enabled es false por defecto: se pide una cantidad fija.

Varias aplicaciones

El gestor reparte los recursos del clúster entre las aplicaciones que corren a la vez.

Un clúster Spark por dentro



El gestor reparte recursos; el driver reparte trabajo.

Quién hace qué en cada nodo

	QUÉ HACE	SE AJUSTA CON
Driver	Ejecuta main(), crea el contexto y reparte tareas	spark.driver.memory y spark.driver.cores
Executor	Ejecuta tareas en varios hilos y guarda datos en memoria o disco	spark.executor.memory, .cores e .instances
Master (standalone)	Gestor incluido con Spark. Puerto 7077; interfaz web en el 8080	SPARK_MASTER_HOST y SPARK_MASTER_PORT
Worker (standalone)	Servicio de cada máquina que lanza executors. Interfaz web en el 8081	SPARK_WORKER_CORES y SPARK_WORKER_MEMORY

Tres gestores y dos modos de despliegue

Standalone

El gestor sencillo que incluye Spark. Bueno para aprender y para clústeres pequeños.

Hadoop YARN

El gestor de recursos de Hadoop 3. Habitual donde ya hay un clúster Hadoop.

Kubernetes

Despliegue de Spark en contenedores sobre un clúster Kubernetes.

DÓNDE CORRE EL DRIVER

Modo cluster

El driver se lanza dentro del clúster.

Modo client

El driver se lanza fuera del clúster, en quien envía la aplicación.

La carpeta conf/ de cada nodo

	PARA QUÉ SIRVE	DÓNDE SE USA
spark-defaults.conf	Propiedades por defecto: una clave y su valor por línea	Quien lanza la aplicación
spark-env.sh	Variables de entorno: IP, puertos, núcleos y memoria del worker	Cada nodo del clúster
workers	Nombres de las máquinas worker, una por línea	El nodo master
log4j2.properties	Nivel y destino de los registros (logs)	Cada nodo
metrics.properties	Sistema de métricas y a dónde se envían	Cada nodo
fairscheduler.xml	Pools del planificador de tipo FAIR	La aplicación

Variables de cada máquina del clúster

bash · conf/spark-env.sh

```
# conf/spark-env.sh (copiar a todos los nodos)
export SPARK_MASTER_HOST=10.0.0.5
export SPARK_MASTER_PORT=7077
export SPARK_WORKER_CORES=8
export SPARK_WORKER_MEMORY=16g
export SPARK_LOCAL_DIRS=/mnt/disk1/spark
```

conf/workers

Un nombre de máquina por línea: worker1.example.com, worker2.example.com…

Arranque

sbin/start-master.sh en el master y sbin/start-workers.sh para los workers.

SPARK_LOCAL_DIRS

Espacio temporal (shuffle, datos en disco): mejor en un disco local rápido.

Valores por defecto de las aplicaciones

properties · conf/spark-defaults.conf

```
# conf/spark-defaults.conf (clave valor)
spark.master spark://10.0.0.5:7077
spark.driver.memory 2g
spark.executor.memory 4g
spark.executor.cores 2
spark.eventLog.enabled true
```

Formato

Una propiedad por línea, con la clave y el valor separados por espacios.

spark-submit

--conf spark.executor.memory=8g pisa lo que diga el fichero.

SparkConf

Lo fijado en el código gana a todo lo demás.

Recursos: dónde y cuánto

	POR DEFECTO	QUÉ CONTROLA
spark.master	Sin valor	A qué gestor se conecta: local[2], spark://host:7077 o yarn
spark.driver.memory	1g	Memoria del proceso driver
spark.driver.cores	1	Núcleos del driver (solo en modo cluster)
spark.executor.memory	1g	Memoria de cada executor
spark.executor.cores	1 en YARN; todos en standalone	Núcleos de cada executor
spark.dynamicAllocation.enabled	false	Sube y baja el número de executors según la carga

Paralelismo y shuffle

	POR DEFECTO	QUÉ CONTROLA
spark.sql.shuffle.partitions	200	Particiones tras un shuffle en joins y agregaciones
spark.default.parallelism	Núcleos totales (mínimo 2)	Particiones por defecto en operaciones de RDD
spark.sql.adaptive.enabled	true desde Spark 3.2	Re-optimiza el plan con estadísticas reales mientras se ejecuta
spark.sql.autoBroadcastJoinThreshold	10 MB	Tabla pequeña que se envía entera a todos los nodos en un join

Memoria, serialización y planificación

	POR DEFECTO	QUÉ CONTROLA
spark.memory.fraction	0.6	Fracción de (memoria – 300 MB) para ejecución y caché
spark.memory.storageFraction	0.5	Parte de esa memoria protegida de desalojo para la caché
spark.serializer	JavaSerializer	Cómo se serializan los datos en red y caché; se recomienda Kryo
spark.eventLog.enabled	false	Guarda eventos para reconstruir la interfaz web después
spark.scheduler.mode	FIFO	Cómo se reparten los recursos entre trabajos: FIFO o FAIR

3.1 · CONFIGURACIÓN: PRECEDENCIA

Dónde se configura y qué gana

1 · En el código

SparkConf o spark.conf.set: gana a todo.

2 · En spark-submit

--conf o --properties-file: pisa el fichero.

3 · En el fichero

spark-defaults.conf: valores base.

python · en un notebook

```
spark.conf.set("spark.sql.shuffle.partitions", "50")
spark.conf.get("spark.sql.shuffle.partitions")
```

En Databricks se fija en la configuración del cómputo; en el cómputo gratuito solo se admiten algunas propiedades.

3.1 · EVALUACIÓN PEREZOSA

Spark no calcula hasta que se lo pides

Transformaciones

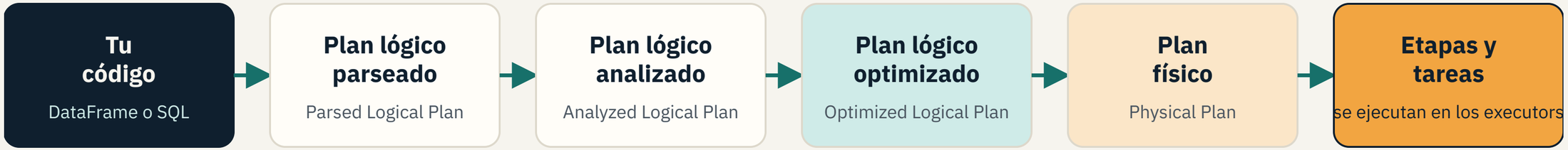
filter, select, groupBy, join, withColumn. Describen qué hacer, pero no ejecutan nada todavía.

Acciones

show, count, display, write. Piden un resultado y es entonces cuando Spark ejecuta todo el plan.

Una celda con solo transformaciones **parece instantánea**: el trabajo llega con la acción.

De tu código al plan de ejecución



Analiza

Comprueba que las columnas y los tipos existen.

Optimiza

Reordena y simplifica el plan sin cambiar el resultado.

Elige cómo

El plan físico decide qué operadores concretos se usan.

Leer un plan con explain()

python

```python
(cambios
   .filter(F.col("wiki") == "eswiki")
   .groupBy("type").count()
   .explain())
```

plan físico · Spark 3.5.3 (resumido)

```
== Physical Plan ==
HashAggregate # conteo final
+- Exchange hashpartitioning(type) # shuffle
   +- HashAggregate # conteo parcial
      +- Filter # wiki = eswiki
         +- FileScan json # PushedFilters:
         EqualTo(wiki,eswiki)
```

explain()

explain() imprime el plan físico; explain(True) añade el parseado, el analizado y el optimizado.

Exchange

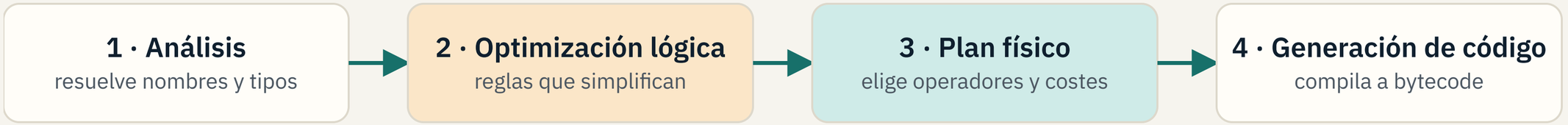
Es el shuffle: mover datos entre máquinas. Es lo caro.

De abajo arriba

El plan se lee desde la lectura del fichero hasta el resultado.

Catalyst: el optimizador de Spark SQL

Entre tu código y las tareas hay un optimizador.



El plan es un árbol; las reglas lo transforman hasta que deja de cambiar.

Reglas de ejemplo

Plegado de constantes, empuje de filtros hacia la lectura, poda de columnas y simplificación de booleanos.

Coste solo en joins

La elección por coste se usa para el algoritmo de join, por ejemplo difundir una tabla pequeña.

Extensible

Se pueden añadir reglas y fuentes de datos propias sin tocar el resto.

Catalyst en acción: antes y después

LO QUE ESCRIBISTE

plan lógico analizado

```
Aggregate [user]
+- Project [wiki, type, user, bot,
            (1 + 1) AS uno]
   +- Filter (wiki = eswiki)
      +- Filter (type = edit)
         +- Project [wiki, type, user, bot]
            +- Relation json (7 columnas)
```

LO QUE DEJA CATALYST

plan lógico optimizado

```
Aggregate [user]
+- Project [user]
   +- Filter (type = edit)
         AND (wiki = eswiki)
         AND type y wiki no nulos
      +- Relation json (7 columnas)
```

Filtros unidos

Los dos filter pasan a una sola condición, con comprobación de nulos añadida.

Columnas podadas

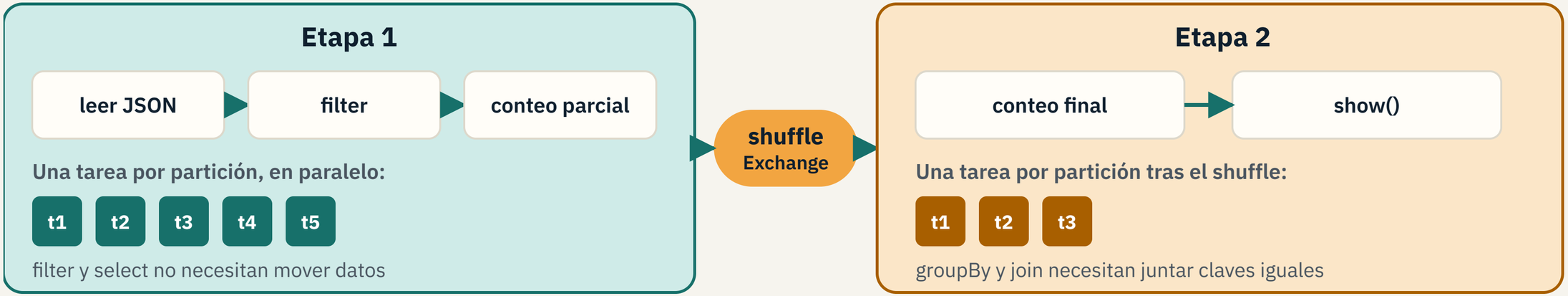
La lectura del JSON pide solo type, user y wiki: 3 de 7 columnas.

Trabajo inútil fuera

La columna (1 + 1) nadie la usa: desaparece del plan.

Job, etapas y tareas: el grafo de ejecución

Acción show() → un job



Cada shuffle parte el trabajo en una nueva etapa.

Dónde se ve: la interfaz web de Spark

Spark dibuja su propio grafo mientras trabaja. Tres pestañas para empezar:

Jobs

Lista de trabajos con su línea de tiempo y el grafo DAG de cada uno.

Stages

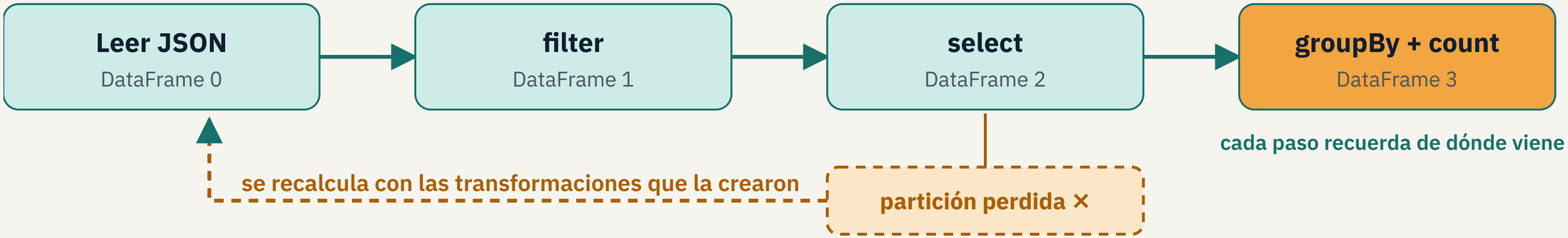
Etapas con sus tareas, y métricas de lectura y escritura de shuffle.

SQL

Grafo de operadores con métricas y los cuatro planes en Details.

Además: **Executors** (recursos de cada nodo) y **Environment** (propiedades aplicadas). En Free Edition (serverless) la Spark UI no está disponible: se usa el perfil de consulta.

Linaje: la receta para recalcular



python · ver el linaje (no funciona en serverless)

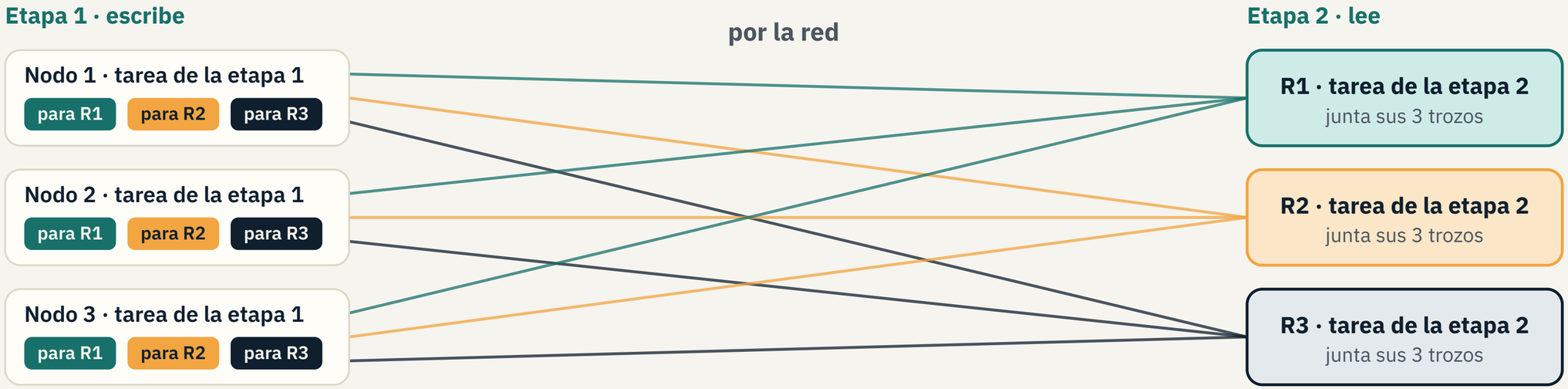
```python
print(cambios.rdd.toDebugString().decode()) # texto con el linaje
```

Dos significados de linaje

	EN SPARK (TÉCNICO)	DEL DATO (GOBIERNO)
Para qué	Recalcular y tolerar fallos	Saber de dónde viene cada dato y quién lo usa
Granularidad	Particiones de un DataFrame	Tablas y columnas
Cuánto dura	Mientras corre la aplicación	Se conserva en el catálogo
Pregunta	¿Cómo recupero esta partición?	¿Qué se rompe si cambio esta columna?
Quién lo usa	El motor, sin que lo veas	Analistas, auditoría y cumplimiento

Shuffle: los datos cambian de nodo

Cada trozo se escribe en el nodo que lo produce.



Se escribe en disco

Cada nodo guarda en su disco local un trozo para cada tarea siguiente.

Se lee por la red

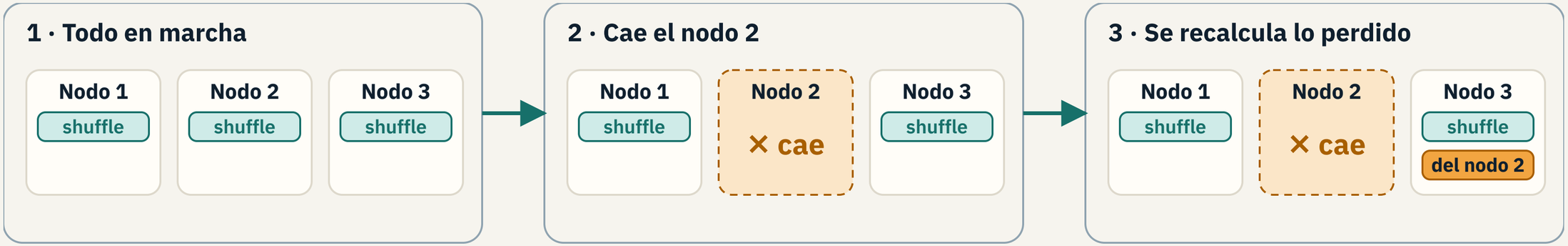
Cada tarea de la etapa 2 pide su trozo a todos los nodos.

Por eso cuesta

Disco y red, y la etapa 2 espera a que acaben todos los de la etapa 1.

Si un nodo cae: no se copia, se recalcula

Spark no copia los datos intermedios: sabe cómo volver a calcularlos.



El driver ve que faltan los trozos del nodo 2 y vuelve a lanzar solo las tareas que los produjeron.

No se copia

Ficheros de shuffle y particiones en memoria: si se pierden, Spark los recalcula con el linaje.

Sí está copiado

Los datos de origen, por el almacenamiento (HDFS o la nube), no por Spark.

Opcional

Persistir con réplica (niveles _2) para no esperar al recálculo; en serverless solo hay df.cache().

Dónde ejecutamos el código

Workspace

El espacio de trabajo donde viven notebooks, ficheros y tablas.

Notebooks

Celdas de Python, SQL o texto, como en Colab, conectadas a Spark.

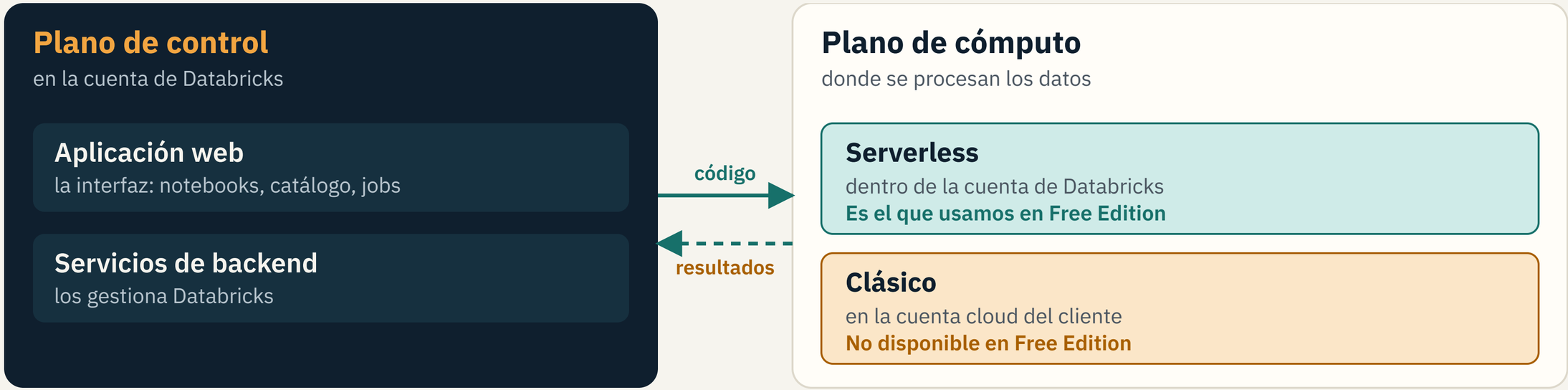
Cómputo

El clúster o servicio que ejecuta el código. En la versión gratuita lo gestiona la plataforma.

Datos y tablas

Subida de ficheros y tablas Delta consultables desde Python o SQL.

Dos planos: control y cómputo



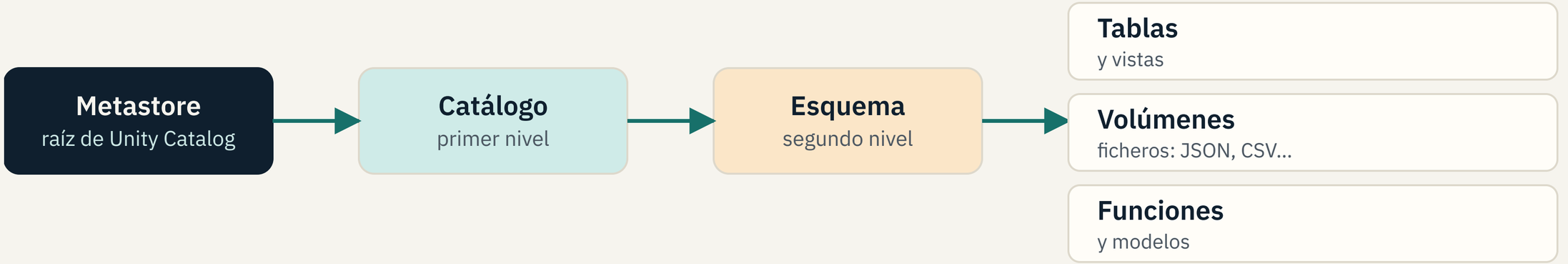
Tú trabajas en la interfaz (control); el código se ejecuta en el plano de cómputo.

Cómputo serverless frente a clásico

Lo que elegimos al crear el cómputo cambia quién lo administra.

	SERVERLESS	CLÁSICO
Dónde corre	Plano de cómputo de la cuenta de Databricks	Cuenta cloud del cliente, en la red virtual del workspace
Quién lo gestiona	Databricks	El cliente: elige y configura los clústeres
En Free Edition	Sí: es el único	No disponible
Streaming	Solo trigger availableNow	Los disparadores de intervalo (según la configuración)
Para qué sirve	Empezar sin administrar nada	Control total del hardware y de la red

Cómo se organizan los datos: Unity Catalog



catalogo.esquema.objeto
workspace.default.wiki_cambios

/Volumes/catalogo/esquema/volumen
/Volumes/workspace/default/streaming

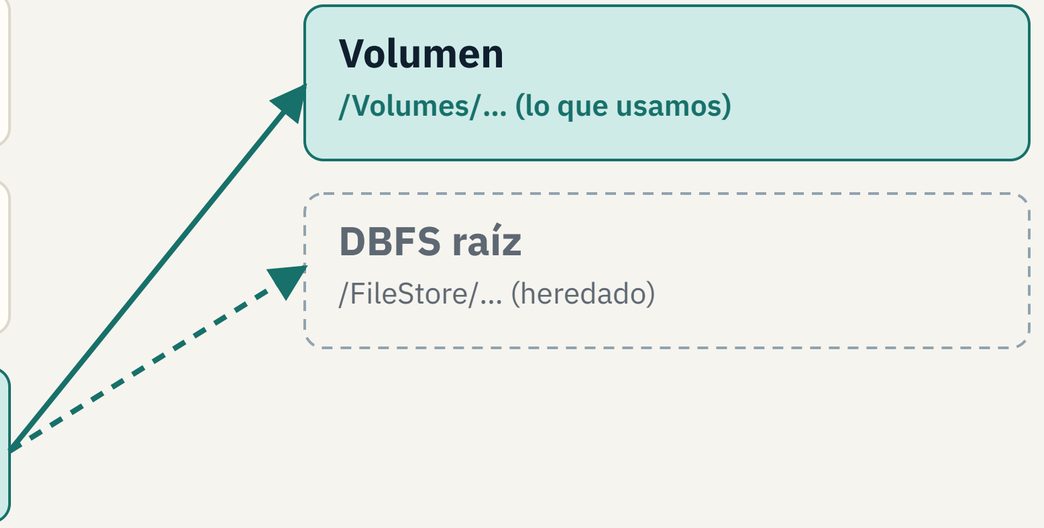
Tablas, volúmenes, funciones y modelos se nombran siempre como catálogo.esquema.objeto.

Dónde queda cada cosa

Sistema de ficheros del workspace
notebooks, consultas, dashboards, repos y librerías

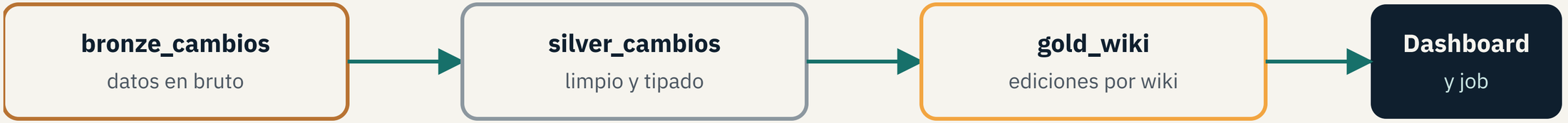
Datos de sistema
resultados de consultas y de jobs, revisiones de notebooks, logs

Tus datos
tablas y volúmenes de Unity Catalog



Para ficheros de datos usamos un volumen; /FileStore es la vía heredada.

Linaje automático con Unity Catalog



tablas → columnas → consumidores: Unity Catalog lo dibuja sin que lo programes

Automático

Captura el linaje de las consultas en Databricks, hasta nivel de columna.

Requisitos

Tablas registradas en Unity Catalog; consultas con DataFrame o SQL.

Dónde verlo

Pestaña Lineage del Catalog Explorer y tablas del sistema.

Ver el linaje con una consulta

python + SQL · Databricks

```
resumen.write.mode("overwrite").saveAsTable("resumen_ediciones_por_wiki")

%sql
SELECT source_table_full_name, target_table_full_name, event_time
FROM system.access.table_lineage
WHERE target_table_full_name LIKE '%resumen_ediciones_por_wiki'
ORDER BY event_time DESC LIMIT 10
```

saveAsTable

Guarda el resultado como tabla del catálogo: así queda registrado el linaje.

system.access

Tablas del sistema con el linaje de tablas y de columnas.

Sin tablas del sistema

Mismo resultado en la pestaña Lineage de la tabla (Catalog Explorer).

3.2 · PREPARACIÓN

Antes de escribir código

1 **Crear la cuenta gratuita**
Registro en Databricks y apertura del espacio de trabajo.

2 **Importar el notebook**
Workspace → Import y elegir 02_BigData_Spark_Databricks_Wikimedia.py (o el .ipynb).

3 **Ejecutar la configuración**
Las dos primeras celdas crean el volumen donde se guardan los datos.

4 **Probar la conexión**
Una celda comprueba que Wikimedia responde con un 200. Sin ficheros que subir.

Capturar el flujo y leerlo como DataFrame

python

```python
capturar(segundos=75) # un fichero JSON nuevo en el volumen

cambios = (spark.read
           .schema(esquema_wiki)
           .json(f"{BASE}/wiki_eventos")
           .withColumn("instante",
               F.to_timestamp(F.from_unixtime("timestamp"))))

print(cambios.count()) # acción: lanza el trabajo
cambios.printSchema() # columnas y tipos
```

capturar()

Función del notebook: guarda unos segundos del flujo en un fichero.

schema

Un esquema explícito evita que Spark lea todo para adivinar los tipos.

instante

Fecha legible a partir del campo timestamp.

Transformar con encadenamiento

```python
(cambios
 .filter((F.col("wiki") == "eswiki") & (F.col("type") == "edit"))
 .select("instante", "title", "user", "bot")
 .show(5, truncate=False))

# Tipos de cambio más frecuentes
(cambios.groupBy("type")
 .count()
 .orderBy(F.desc("count"))
 .show())
```

F.col()

Se refiere a una columna dentro de una expresión.

&

Une condiciones; cada una entre paréntesis.

show()

Es la acción que dispara el cálculo y muestra filas.

¿Cuánta actividad es de bots?

python

```
(cambios.groupBy("wiki")
 .agg(F.count("*").alias("cambios"),
      F.sum(F.when(F.col("bot") == True, 1)
             .otherwise(0)).alias("de_bots"))
 .withColumn("bots_%",
             F.round(F.col("de_bots") / F.col("cambios") * 100, 1))
 .filter(F.col("cambios") ≥ 20)
 .orderBy(F.desc("cambios"))
 .show(10, truncate=False))
```

agg

Varias agregaciones a la vez sobre cada grupo.

when / otherwise

Condicional por fila: cuenta 1 si lo hizo un bot, 0 si no.

filter tras agg

Equivale al HAVING de SQL: descarta proyectos con pocos cambios.

Unir con una tabla pequeña

python

```python
idiomas = spark.createDataFrame([
    ("enwiki", "inglés"), ("eswiki", "español"),
    ("frwiki", "francés"), ("dewiki", "alemán"),
], "wiki string, idioma string")

(cambios
 .join(F.broadcast(idiomas), on="wiki", how="left")
 .withColumn("idioma", F.coalesce("idioma", F.lit("otros")))
 .groupBy("idioma")
 .agg(F.count("*").alias("cambios"))
 .orderBy(F.desc("cambios"))
 .show())
```

Clave

wiki está en las dos tablas: es la clave del join, igual que en SQL.

broadcast

La tabla pequeña se envía entera a cada máquina: no se mueve la grande.

left

Conserva los cambios de proyectos que no están en la tabla (quedan como otros).

Los tres usuarios más activos de cada proyecto

python

```python
from pyspark.sql.window import Window

por_usuario = (cambios.filter(F.col("bot") == False)
               .groupBy("wiki", "user")
               .agg(F.count("*").alias("cambios")))

ventana = Window.partitionBy("wiki").orderBy(F.desc("cambios"))

(por_usuario
 .withColumn("puesto", F.row_number().over(ventana))
 .filter(F.col("puesto") <= 3)
 .orderBy("wiki", "puesto")
 .show(truncate=False))
```

Ventana

Calcula algo dentro de cada grupo sin perder las filas, por ejemplo un ranking por proyecto.

partitionBy

Define el grupo: cada proyecto tiene su propio ranking.

row_number

Numera las filas de cada grupo según el orden.

Lo mismo con SQL

python

```
cambios.createOrReplaceTempView("cambios")
idiomas.createOrReplaceTempView("idiomas")

%sql
SELECT COALESCE(i.idioma, 'otros') AS idioma,
       COUNT(*) AS cambios,
       SUM(CASE WHEN c.bot THEN 1 ELSE 0 END) AS de_bots
FROM cambios c
LEFT JOIN idiomas i ON c.wiki = i.wiki
GROUP BY COALESCE(i.idioma, 'otros')
ORDER BY cambios DESC
LIMIT 10
```

Vista temporal

Registra el DataFrame con un nombre para poder consultarlo con SQL.

Mismo motor

SQL y DataFrame producen el mismo plan de ejecución.

Cache: útil, pero no es gratis

Memoria de un executor: una región compartida



Si la ejecución necesita sitio, puede expulsar parte de la caché.

Cuándo ayuda

Un DataFrame caro que se usa en varias acciones. La primera lo calcula y guarda; las siguientes lo leen.

Cuesta memoria

La caché y la ejecución comparten memoria. Cachear mucho deja menos para ejecutar: más spill o fallos.

Si no cabe

df.cache() usa memoria y disco. Leer del disco puede costar lo mismo que recalcular.

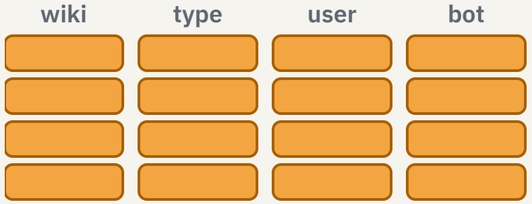
Libérala

unpersist() cuando ya no se use. Si no, Spark la expulsa solo cuando necesita sitio.

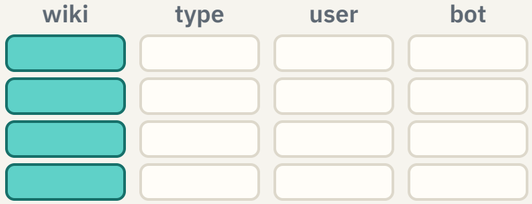
Cachea solo lo que reutilizas y cabe. Cachear «por si acaso» puede dejar el job más lento.

Parquet: columnas y compresión

Por filas: lees todo



Por columnas: solo wiki



python · escribir y leer Parquet

```
df.write.mode("overwrite").parquet(f"{BASE}/salida")

spark.read.parquet(f"{BASE}/salida")
  .groupBy("wiki").count().show()
```

Por columnas

Los datos de una columna van juntos: se leen solo las que hacen falta y comprimen mejor.

Lleva el esquema

Los ficheros son autodescriptivos: al leerlos recuperas columnas y tipos.

Compresión

snappy por defecto; se cambia con la opción compression.

Avro: filas y esquema



Cada registro se guarda entero, con el esquema al principio.

python · Avro (necesita el módulo spark-avro)

```
# módulo externo: --packages org.apache.spark:spark-avro_2.12:3.5.3
df.write.format("avro").save(f"{BASE}/salida_avro")

spark.read.format("avro").load(f"{BASE}/salida_avro")
```

Por filas

Cada registro va entero: va bien para escribir y leer registros completos, como en un flujo.

Esquema evolutivo

El esquema viaja con los datos y admite cambios controlados. Muy usado con Kafka.

En Spark

Módulo externo; format("avro"); compresión snappy por defecto.

Delta: Parquet con registro de transacciones

Ficheros Parquet
los datos

Registro de cambios
qué ficheros forma cada versión



python y SQL · tabla Delta

```
resumen.write.format("delta").mode("overwrite")
  .saveAsTable(f"{CATALOGO}.{ESQUEMA}.resumen")

DESCRIBE HISTORY resumen -- versiones de la tabla
```

Transacciones

Las escrituras son atómicas: nadie ve datos a medias, y los lectores ven una versión coherente.

Versiones

Cada escritura crea una versión: se puede ver el historial y consultar una anterior.

Por defecto

En Databricks, cualquier tabla nueva es Delta salvo que digas otra cosa.

Cuidado: las salidas salen en trozos

lo que ves tras df.write.parquet("salida")

```
salida/
  _SUCCESS
  part-00000-…-c000.snappy.parquet
  part-00001-…-c000.snappy.parquet
  part-00002-…-c000.snappy.parquet
```

python · controlar los ficheros

```
df.coalesce(1).write.parquet(ruta_uno)
df.write.partitionBy("wiki").parquet(ruta)
```

Es una carpeta

Spark escribe una carpeta con un fichero por partición, más _SUCCESS. Se lee la carpeta entera.

Tantos ficheros como particiones

Si salen demasiados, son ficheros pequeños; con coalesce(1) sale uno, pero escribe una sola tarea.

partitionBy crea carpetas

wiki=enwiki, wiki=eswiki…: bien con pocos valores; con muchos, miles de carpetas.

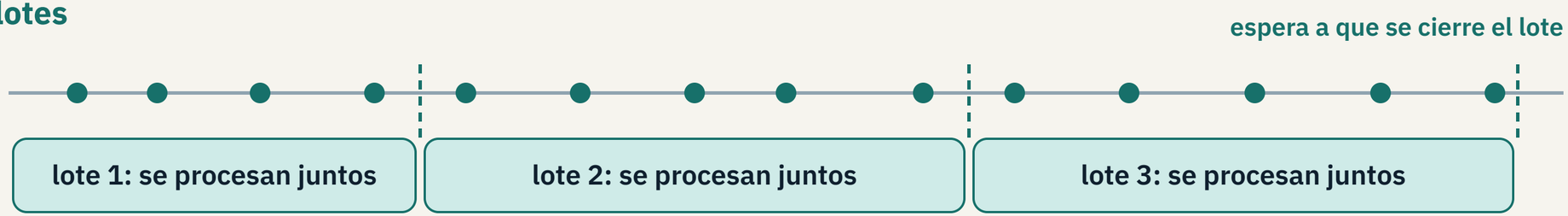
De lotes a flujo

Spark trata el flujo como una tabla que crece: la consulta no cambia, cambia cómo se alimenta.

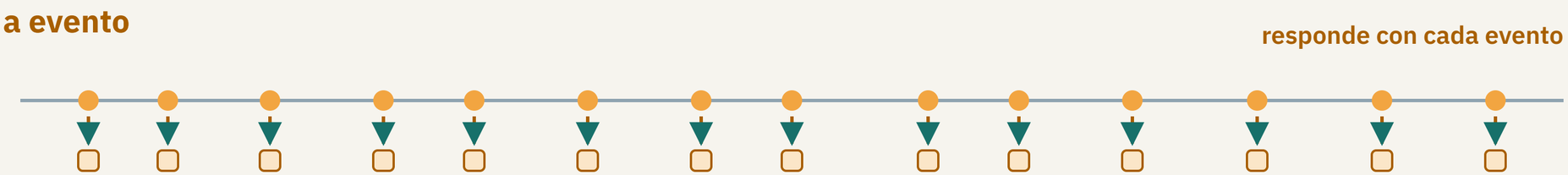
	LOTES (BATCH)	FLUJO (STREAMING)
Los datos	Un conjunto cerrado, con principio y fin	Llegan sin parar; no hay final
El resultado	Se calcula una vez, al terminar	Se actualiza cada vez que llega algo
En Spark	spark.read	spark.readStream
La consulta	DataFrame o SQL	La misma: DataFrame o SQL
Cómo lo ejecuta	De una vez	En micro-lotes sobre una tabla que no deja de crecer

Micro-lotes frente a evento a evento

Spark · micro-lotes



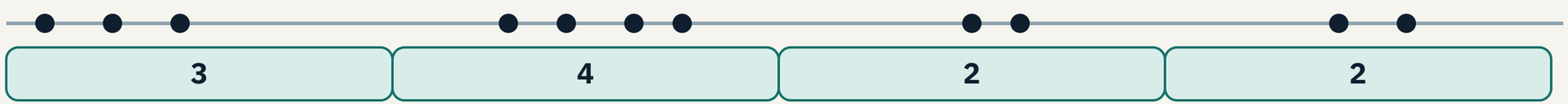
Flink · evento a evento



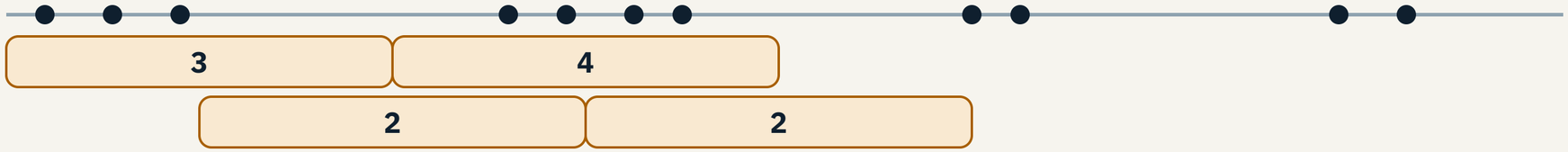
Los puntos son los mismos eventos: Spark los reúne y los procesa en bloque; Flink los procesa uno a uno.

Ventanas: tres formas de agrupar el tiempo

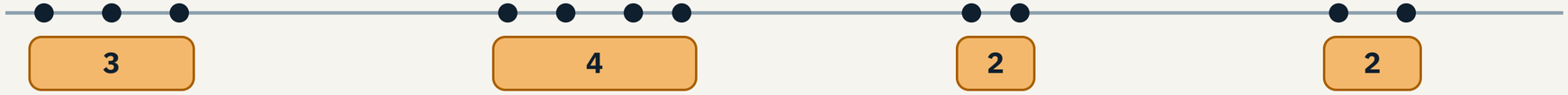
Fija
cada 400 sin solaparse



Deslizante
tamaño 400, avanza 200



De sesión
se cierra tras un silencio



Mismos eventos, cuentas distintas según el tipo de ventana.

Spark Structured Streaming y Flink

Dos formas de resolver el mismo problema: cuánta espera aceptas y cuánta simplicidad quieres.

	SPARK STRUCTURED STREAMING	FLINK
Procesamiento	Micro-lotes por defecto	Evento a evento
Latencia	~100 ms en el mejor caso; ~1 ms en modo continuo (experimental)	Pensado para baja latencia
Garantías	Exactly-once en micro-lotes; at-least-once en el modo continuo	Con checkpoints (a comprobar)
Lotes	Motor propio de lotes y de DataFrames	Datos acotados como un flujo finito
En el curso	Lo usamos en Databricks	Solo como contexto

Qué flujo usamos y por qué

Twitter / X

La lectura de posts es de pago y el streaming en tiempo real, solo para planes de empresa. No se puede reproducir en clase.

Wikimedia EventStreams

Cambios de Wikipedia en directo, gratis y sin clave. Se leen por HTTP en formato Server-Sent Events.

Probado: desde Databricks Free en Europa, el dominio de Wikimedia responde con un 200.

El recorrido completo



Python captura el flujo en ficheros; Spark los procesa como un flujo y solo lee lo nuevo.

Capturar el flujo en ficheros

python · captura

```
URL = "https://stream.wikimedia.org/v2/stream/recentchange"
cabeceras = {"User-Agent": "CursoBigData/1.0 (tu-correo)"}

with requests.get(URL, headers=cabeceras, stream=True) as r, open(ruta, "w") as f:
    for linea in r.iter_lines():
        if linea.startswith(b"data:"):
            f.write(linea[5:].decode() + "\n")
            n += 1
        if n ⩾ 2000: break
```

stream=True

No espera a que acabe la respuesta: lee según llega.

data:

Cada evento llega en una línea que empieza por data:.

Un fichero por captura

Cada captura es un trozo nuevo para Spark.

Structured Streaming sobre la carpeta

python · Spark

```python
flujo = (spark.readStream
         .schema(esquema_wiki)
         .json(f"{BASE}/wiki_eventos"))

(flujo.writeStream
      .option("checkpointLocation", f"{BASE}/checkpoint_wiki")
      .trigger(availableNow=True)
      .toTable(TABLA_WIKI))
```

Esquema

Se declara a mano: un flujo no puede mirar todos los datos para inferirlo.

Checkpoint

Recuerda lo ya procesado: al relanzar solo lee lo nuevo.

availableNow

En serverless es el disparador recomendado; los de intervalo dan error.

Verlo crecer, ronda a ronda

python · demo en vivo

```python
for ronda in range(1, 7):
    capturar(segundos=10) # llegan datos nuevos
    lanzar(leer_wiki(), "wiki_cambios", "copia") # Spark lee solo lo nuevo
    lanzar(por_tipo, "wiki_por_tipo", "por_tipo", modo="complete")
    total = spark.table("wiki_cambios").count()
    print(f"Ronda {ronda} · {time.strftime('%H:%M:%S')} · total {total}")
```

Cada ronda

Una captura nueva de 10 segundos: llegan datos.

Solo lo nuevo

El checkpoint evita releer lo anterior.

Qué verás

El total y el porcentaje de bots cambian en cada ronda.

Caso 2: filtrar el flujo

python · Spark

```python
humanas = (leer_wiki()
           .filter((F.col("type") == "edit") & (F.col("bot") == False)
                   & F.col("wiki").isin("enwiki", "eswiki", "frwiki"))
           .select("instante", "wiki", "title", "user"))

lanzar(humanas, "wiki_ediciones_humanas", "humanas")
```

Sin estado

Cada evento se mira por separado: no hay que recordar nada.

Modo append

Solo se añaden filas nuevas a la tabla.

lanzar()

Función del notebook: escribe en Delta con su checkpoint.

Caso 3: contar con estado

python · Spark

```
por_tipo = (leer_wiki()
            .groupBy("type", "bot")
            .agg(F.count("*").alias("cambios")))

lanzar(por_tipo, "wiki_por_tipo", "por_tipo", modo="complete")
```

Con estado

Spark recuerda los conteos entre lotes.

Modo complete

En cada lote se reescribe el resultado completo.

Pregunta

¿Qué proporción de los cambios hacen los bots?

Caso 4: ventanas de tiempo

python · Spark

```python
por_minuto = (leer_wiki()
              .groupBy(F.window("instante", "1 minute"))
              .agg(F.count("*").alias("cambios")))

lanzar(por_minuto, "wiki_por_minuto", "por_minuto", modo="complete")
```

window

Agrupa los eventos por tramos de la hora del evento.

Serie temporal

El flujo pasa a ser cambios por minuto, listo para un gráfico.

Captura larga

Hay que capturar más de un minuto para ver varias ventanas.

Varias consultas, una sola fuente

Cuatro consultas sobre la misma carpeta, cada una con su checkpoint.

Copia

Guarda todo lo que llega, con sus tipos. Es el bronce.

Filtro

Solo ediciones humanas en tres Wikipedias.

Conteo

Cambios por tipo y por bot, con estado.

Ventana

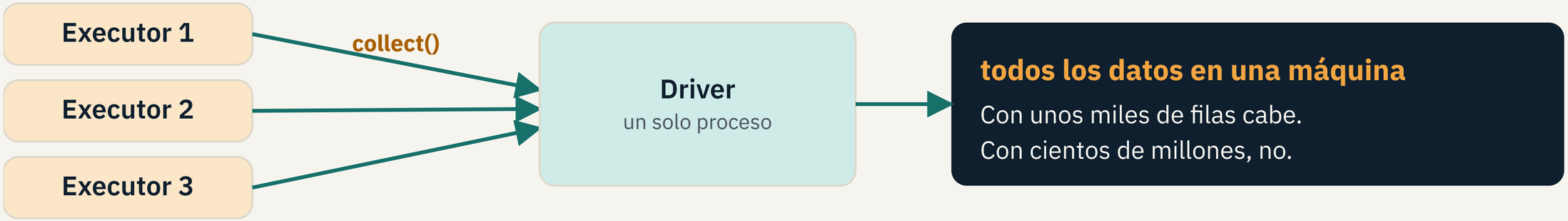
Cambios por minuto, como serie temporal.

Seis problemas que se repiten

Casi todos aparecen al pasar de pocos datos a muchos.

	CÓMO SE NOTA	PRIMERA MEDIDA
collect() de más	El driver se queda sin memoria	limit(), show() o agregar antes
Particiones mal ajustadas	Tareas enormes o miles de minúsculas	Ajustar el shuffle o dejar actuar a AQE
Desbalanceo (skew)	Una tarea tarda mucho más que el resto	AQE para joins, salting, rebalance
Falta de memoria	Spill a disco o executors que caen	Más paralelismo o más memoria por núcleo
Código que no escala	UDF, ventana sin partición, broadcast enorme	Funciones nativas y partitionBy
Recalcular sin cache	La misma lectura se repite en cada acción	cache() de lo que se reutiliza

collect(): todos los datos al driver



PELIGROSO

python · trae todo al driver

```
filas = cambios.collect()
pdf = cambios.toPandas()
```

SEGURO

python · trae poco o nada

```
cambios.limit(20).show()
resumen = cambios.groupBy("wiki").count().collect()
resumen.write.saveAsTable("resumen")
```

Variables que no se comparten

PELIGROSO

python · una variable fuera de la función

```python
total = 0
def sumar(fila):
    global total
    total += 1 if fila["bot"] else 0
cambios.foreach(sumar)
print(total) # no es lo que esperas
```

SEGURO

python · que sume el clúster

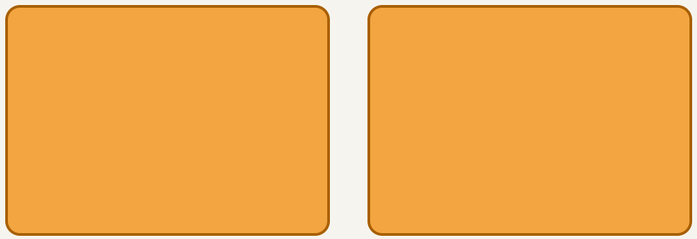
```python
cambios.agg(F.sum(F.when(F.col("bot"),
1).otherwise(0))).show()

# o, para quedarte con el valor:
n_bots = cambios.agg(F.sum(F.when(F.col("bot"),
1).otherwise(0))).first()[0]
```

Cada executor recibe su copia de la variable: el total del driver no cambia.

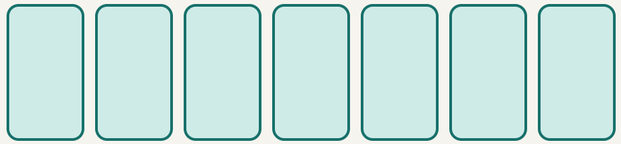
Particiones: ni pocas ni demasiadas

Pocas particiones



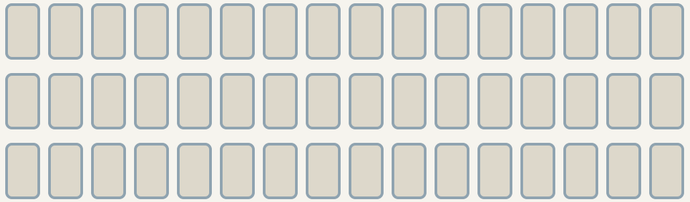
pocas tareas, muy grandes

Ajustadas



varias tareas por núcleo

Demasiadas



mucho reparto y poco trabajo

Pocas: spill u OOM

Cada tarea maneja demasiados datos y le falta memoria.

Muchas: coste de gestión

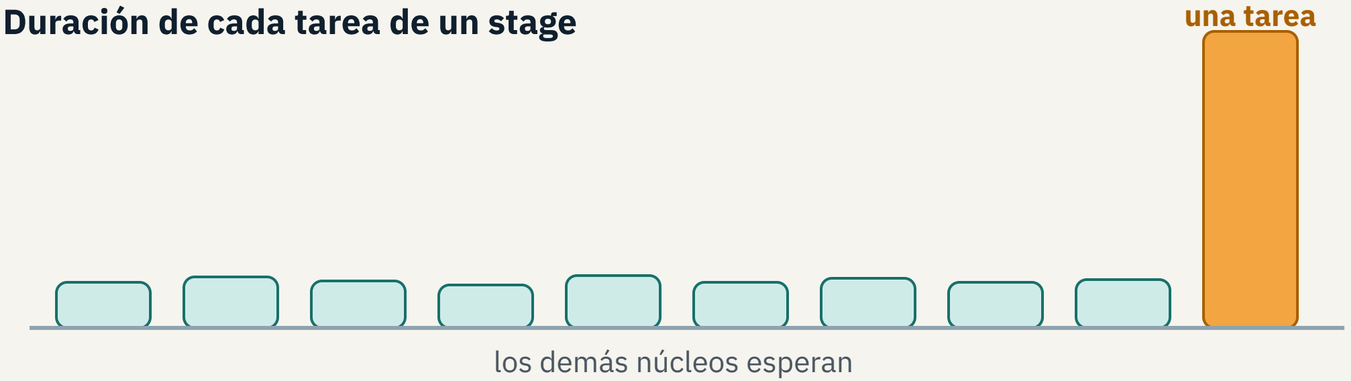
Cada tarea tiene un coste fijo; con trozos minúsculos pesa más que el trabajo.

Punto de partida

Spark recomienda 2-3 tareas por núcleo; en SQL, 200 particiones de shuffle por defecto.

Desbalanceo: una tarea que no acaba

Duración de cada tarea de un stage



En el notebook (12.3)

Mira qué % de los cambios
tiene el proyecto con más cambios
Si es muy alto, hay desbalanceo

groupBy('wiki') lo manda a una tarea

Cómo se detecta

Máximo de la duración de tareas muy por encima del percentil 75 (más de un 50 %).

Qué hace Spark

AQE reparte las particiones sesgadas de un join automáticamente (activado por defecto).

Qué podemos hacer

Salting (añadir un trozo aleatorio a la clave) o reequilibrar antes de escribir.

Falta de memoria: del spill al OOM

Memoria de ejecución de un executor



Spill

Spark pasa datos de memoria a disco cuando le falta memoria de ejecución (shuffles, joins, ordenaciones, agregaciones). Funciona, pero más lento.

Executor sin memoria

Se eliminan los executors y se reintentan sus tareas; si fallan varias veces, falla el job.

Cómo comprobarlo

Probar con el doble de memoria por núcleo. Importa la relación memoria/núcleo, no la memoria total.

Tres trampas en el propio código

Broadcast

Peligroso

```
grande.join(F.broadcast(otra_grande), "k")
```

Mejor

dejar que Spark decida (umbral por defecto: 10 MB) y solo difundir tablas pequeñas

UDF de Python

Peligroso

```
@F.udf("double")
def minusculas(t): ...
```

Mejor

```
F.lower(F.col("title"))
```

Ventana

Peligroso

```
Window.orderBy("timestamp")
```

Mejor

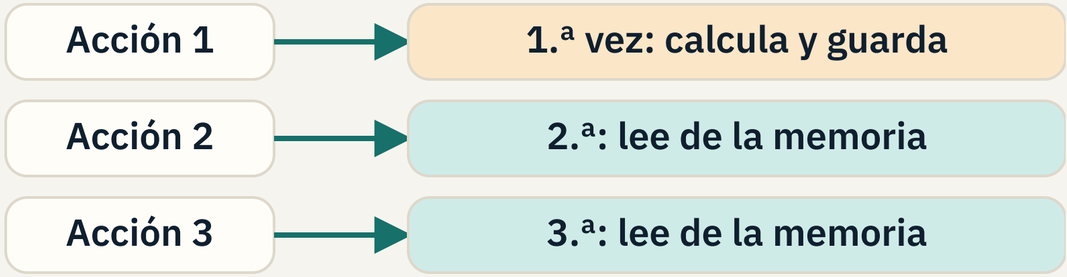
```
Window.partitionBy("wiki").orderBy("timestamp")
```

Recalcular sin necesidad

Sin cache



Con cache



Perezoso

Cada acción ejecuta todo el plan desde el principio.

cache() y persist()

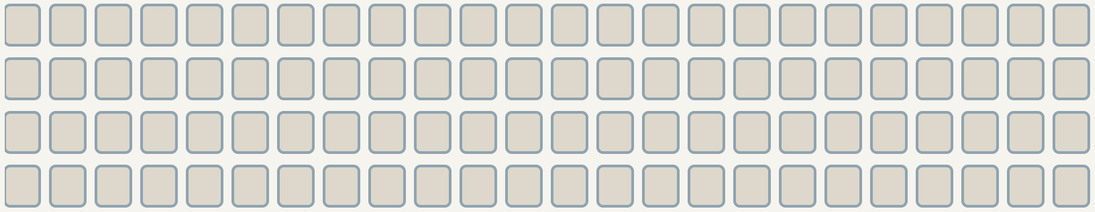
Guardan el resultado en memoria la primera vez; unpersist() lo libera.

En serverless

df.cache() sí; cacheTable(), CACHE TABLE y checkpoint() dan error.

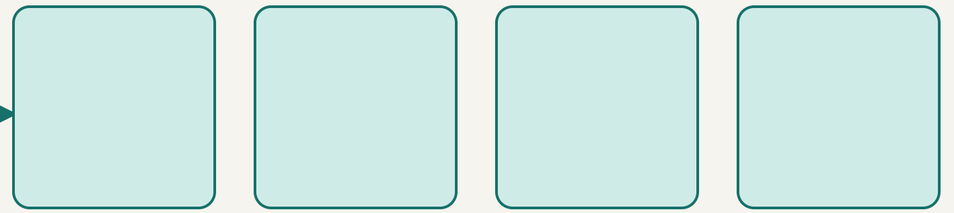
El problema de los ficheros pequeños

Muchos ficheros pequeños



cada fichero hay que listarlo y abrirlo

Pocos ficheros grandes



Spark los reparte en trozos de hasta 128 MB

De dónde salen

De escrituras frecuentes y pequeñas: nuestras capturas de streaming crean un fichero por captura.

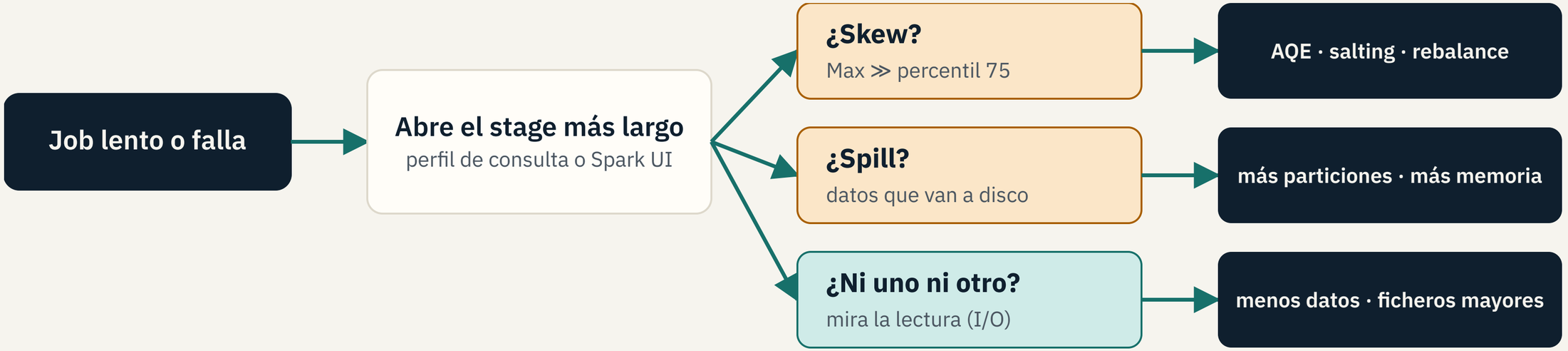
Cuánto cuestan

Spark estima 4 MB de coste por abrir cada fichero y agrupa ficheros hasta 128 MB por partición.

Qué hacer

Escribir menos ficheros más grandes (compactar) en vez de miles pequeños.

Cómo diagnosticar un job lento



Primero mide en qué stage y por qué tarda; después elige la medida.

3.6 · CRITERIO

¿Cuándo compensa un clúster?

Los datos no caben

Superan la memoria de una máquina o tardan demasiado en procesarse.

Hay prisa

El tiempo de proceso importa y se puede repartir el trabajo.

Muchas fuentes

Hay que unir y limpiar datos de muchos orígenes de forma repetible.

Equipos y gobierno

Varias personas comparten datos, permisos y tablas.

EJERCICIO · 20 MINUTOS

Ahora te toca a ti

1. Calcula el porcentaje de cambios hechos por bots en cada proyecto y muestra los cinco con mayor porcentaje (con al menos 20 cambios).
2. ¿Cuántos cambios de cada tipo hay en cada ventana de un minuto? Usa el caso de ventanas de streaming como modelo.
3. Compara el tiempo de una agregación con y sin cache().
4. Haz una nueva captura y relanza el caso 1 de streaming: ¿cuántas filas nuevas entran y por qué no se releen las anteriores?

SOLUCIONES DEL EJERCICIO (1 DE 4)

Solución 1: bots por proyecto

python

```python
(cambios.groupBy("wiki")
 .agg(F.count("*").alias("cambios"),
      F.sum(F.when(F.col("bot"), 1)
             .otherwise(0)).alias("de_bots"))
 .filter(F.col("cambios") ≥ 20)
 .withColumn("bots_%",
     F.round(F.col("de_bots") / F.col("cambios") * 100, 1))
 .orderBy(F.desc("bots_%"))
 .limit(5)
 .show())
```

Qué cambia

Es la agregación de la sección 6, pero se ordena por bots_% y no por cambios.

Filtro de 20

Sin él, un proyecto con 2 cambios y 1 de un bot saldría primero con un 50 %.

limit(5)

Se queda con los cinco primeros del orden.

SOLUCIONES DEL EJERCICIO (2 DE 4)

Solución 2: tipos por ventana de un minuto

```python
por_tipo_min = (leer_wiki()
    .groupBy(F.window("instante", "1 minute"), "type")
    .agg(F.count("*").alias("cambios")))

lanzar(por_tipo_min, "wiki_tipo_minuto", "tipo_minuto",
       modo="complete")

display(spark.table(f"{CATALOGO}.{ESQUEMA}.wiki_tipo_minuto")
    .select(F.col("window.start").alias("minuto"),
            "type", "cambios")
    .orderBy("minuto", "type"))
```

Ventana y tipo

El groupBy lleva la ventana y type: un conteo por minuto y por tipo.

complete

En cada lote se reescribe el resultado completo, igual que en el caso 11.5.

Sin streaming

Con cambios en lugar de leer_wiki() el mismo groupBy funciona por lotes.

SOLUCIONES DEL EJERCICIO (3 DE 4)

Solución 3: con y sin cache

python

```python
def tiempo(f):
    t = time.time(); f(); return round(time.time() - t, 2)

base = cambios.filter(F.col("type") == "edit")
agregar = lambda df: df.groupBy("wiki").count().collect()

con = base.cache()
con.count() # materializa la cache

print("Sin cache:", tiempo(lambda: agregar(base)), "s")
print("Con cache:", tiempo(lambda: agregar(con)), "s")

con.unpersist() # libera la memoria
```

Misma consulta

Se mide la misma agregación sobre el DataFrame sin cache y sobre el cacheado.

Materializar antes

El count() carga la cache; esa carga no entra en la medida.

Pocos datos

Con miles de filas la diferencia es pequeña: crece con el volumen y con cada reutilización.

SOLUCIONES DEL EJERCICIO (4 DE 4)

Solución 4: el checkpoint evita releer

python

```python
tabla = f"{CATALOGO}.{ESQUEMA}.wiki_cambios"
antes = spark.table(tabla).count()

capturar(segundos=20) # un fichero nuevo
lanzar(leer_wiki(), "wiki_cambios", "copia")

despues = spark.table(tabla).count()
print("Filas nuevas:", despues - antes)
```

Solo lo nuevo

Entran las filas del fichero nuevo, no las de las capturas anteriores.

Por qué

El checkpoint (ck_copia) recuerda qué ficheros ya procesó.

Coste

Cada ejecución cuesta lo que llega de nuevo, no todo el histórico.

CIERRE

Lo que nos llevamos

- Spark reparte los datos entre máquinas y combina los resultados.
- Es perezoso: las transformaciones no calculan hasta una acción.
- La lógica de pandas y SQL se traslada casi directamente a PySpark.
- Un flujo es una tabla que crece: la misma consulta, procesada solo con lo nuevo.

SIGUIENTE BLOQUE

Visualización y BI

Cuadros de mando con Plotly y panorama de herramientas de BI.